Mayalok: Smartphone-to-Web 3D Digital Twin Pipeline

Techfest 2026-27, IIT Bombay | Techfest Namma Space challenge, Round 1: 3D Reconstruction & Roaming [Team ID: ____]

1. Abstract

Mayalok is an automated pipeline that turns ordinary smartphone videos and photos of an indoor space into a photorealistic, metric-scale 3D Gaussian Splat and serves it as a zero-install first-person web walkthrough. A user follows a short capture SOP; one command (`namma build`) then extracts sharp keyframes, recovers camera poses with COLMAP, fixes metric scale and gravity using printed ArUco markers or a fixed phone-height rule, trains a 3D Gaussian Splatting model with gsplat, cleans it, and derives a 5 cm walkable occupancy grid and minimap. The result is packaged as a compressed `.spz` splat plus a scene manifest and rendered in the browser with React, Three.js and Spark.js. Round 1 deliverables (SOP, reconstruction, free roaming) were exercised on a real site, an incubation centre, captured on 24 Oct 2026 with 8 smartphone videos and about 40 photos. The same grid also powers POI search and A* routing already present in the viewer, which are the Round 2 deliverables.

2. System Architecture

```
Smartphone videos + photos
           |
           v
[1] namma.frames    sharpest-frame keyframe extraction (ffmpeg, Laplacian variance)
           |
           v
[2] namma.sfm       COLMAP / pycolmap: SIFT features, sequential matching, global mapper
           |
           v
[3] namma.align +   gravity "up" from cameras, floor plane fit, metric scale
    namma.markers   (ArUco markers, else 1.35 m phone-height SOP), yaw alignment
           |
           v
[4] namma.train     gsplat simple_trainer, MCMC densification, 30k steps, <=1.5M Gaussians
           |         (Brush as Windows fallback trainer)
           v
[5] namma.clean     opacity + statistical outlier removal, floor levelling to y = 0
           |
           v
[6] namma.navgrid   5 cm occupancy grid, clearance map, top-down minimap
           |
           v
[7] namma.export    scene.spz (PlayCanvas splat-transform) + scene.json manifest
           |
           v
[8] Web viewer      React + Three.js + Spark.js, WASD / pointer-lock first person,
                    POIs, Fuse.js search, A* in a Web Worker
```

Stages are checkpointed by the orchestrator (`namma.pipeline`), so a failed or tuned stage resumes without recomputing earlier ones (`--force` recomputes all). Each stage reads and writes plain files, so any stage can be swapped independently.

3. Pipeline Design

3.1 Keyframe extraction (`namma.frames`). Videos are sampled at 3x the target rate (default 2 keyframes per second after selection, long side capped at 1600 px). In each window only the sharpest frame is kept, measured as variance of the Laplacian on a downscaled grayscale copy; windows below 35% of the median sharpness are

dropped. This removes most motion blur from handheld walking. Photos are kept unless blurry. Each clip gets its own folder, and a minimum-frame check fails the build early on unusable footage.

3.2 Structure from Motion (`namma.sfm`). pycolmap (COLMAP 4.x) extracts SIFT features with a SIMPLE_RADIAL camera model, then matches each frame against its next 15 neighbours (sequential matching), with an optional vocabulary tree for loop closure. Poses are recovered with COLMAP's global mapper (GLOMAP) and fall back to incremental mapping if too few frames register. Images are then undistorted into a gsplat-ready dataset.

3.3 Metric scale and gravity alignment (`namma.align`, `namma.markers`). SfM output has arbitrary scale and orientation. "Up" is estimated from camera orientations, and the floor plane is found as the lowest dense layer of points below camera height, refined by RANSAC (tilt limited to 10 degrees). Scale priority is: manual override, then **ArUco markers** (`DICT_4X4_50`, 15 cm side, detected in up to 400 frames, triangulated by DLT, accepted only with at least 3 views and low reprojection error; scale is the printed side divided by the median reconstructed side), then the **phone-height SOP** (1.35 m camera height above the detected floor). Yaw is aligned to the dominant wall directions (Manhattan yaw from x/z histograms). The scene is left in a metric, floor-at-zero, y-up frame.

3.4 Training (`namma.train`). gsplat's reference trainer (3DGS, Kerbl et al. 2023) runs for 30,000 steps with MCMC densification and a hard cap of 1.5 million Gaussians, which bounds GPU memory and download size. World-space normalisation is disabled so the metric frame is preserved, and every 8th frame is held out to report PSNR. On Windows machines where gsplat does not build, Brush is used as a fallback trainer.

3.5 Cleaning (`namma.clean`). Gaussians with opacity below 0.05 are removed, then statistical outliers (8 nearest neighbours, distance above mean plus 3 standard deviations) are removed as floaters. The floor height is re-fitted from the dense splat layer near y = 0 and corrected with a vertical-only shift, so view-dependent colour is unaffected.

3.6 Navigation grid (`namma.navgrid`). The cleaned splat is sliced into a 5 cm x 5 cm occupancy grid on the floor plane. A cell is walkable if floor was reconstructed there, nothing solid lies between 0.12 m and 1.9 m height, and it is at least 0.25 m (agent radius) from any obstacle; cells the camera walked through are marked free. A clearance map and a top-down minimap image are produced. This grid is the single spatial index for collision, pathfinding and AR.

3.7 Export (`namma.export`). The splat is converted to compressed `.spz` (v3) with PlayCanvas splat-transform, and `scene.json` records the frame convention, bounds, spawn pose, AR anchors (marker positions) and QC statistics.

3.8 Alternative capture route. Users can instead capture with Polycam, Luma AI or Scaniverse, export a `.ply` splat and run `namma process-splat`, which skips stages 1-4 and performs cleaning, navgrid and export.

4. Capture SOP Summary

Full checklist: `docs/SOP.md`.

- **Lighting and scene:** all lights on, no glare; clear floor clutter; close unwanted doors; avoid mirrors and glass.
- **Scale:** 3-4 printed ArUco markers (15 cm, `DICT_4X4_50`) taped flat on the floor, or a constant 1.35 m phone height.
- **Motion:** slow walking, perimeter loop plus inner pass, corner shots into every corner, ceiling and floor sweeps, 60-70% overlap, consistent orientation, no in-place spins.
- **Rooms:** continuous video through doorways looking back then forward for small spaces; one clip per zone with about 2 m overlap for large ones; walk the real routes people use.
- **Effort:** about 3-5 minutes per room.

5. Web Viewer Technical Approach

- **Stack:** Vite, React 19, Three.js, **Spark.js** (GPU Gaussian splat renderer for Three.js, reads `.spz`), Fuse.js for fuzzy search.
- **Movement:** first-person controls (`PlayerControls`) with **WASD**, pointer-lock mouse look (drag-look and touch supported), Shift to run, Space/C vertical movement in noclip mode. Eye height is 1.6 m and walk speed 1.6 m/s, both frame-rate independent via delta time. Free roaming works with no navigation active, as required.
- **Scene loading:** a scene folder holds `scene.spz`, `scene.json`, the navgrid and the minimap; the loader streams the compressed splat into Spark and positions the player at the manifest spawn pose.
- **Navigation layer (beyond Round 1):** POIs and zone polygons live in `pois.json`; Fuse.js search jumps to a POI; A* over the navgrid runs in a **Web Worker** so the UI stays responsive, producing turn-by-turn directions, a path line and an autopilot walk. An admin mode (`?admin`) edits POIs and zones.
- **Compatibility and hosting:** WebGL2 browsers, no installation; built as a static site and deployed to static hosting.

6. Performance Notes

- `.spz` compression and the 1.5M Gaussian cap keep downloads small and GPU memory bounded; room-scale scenes load fast and render smoothly on integrated GPUs.
- Spark sorts and renders splats on the GPU, and the A* search runs off the main thread, so camera input is never blocked.
- Keyframe selection (about 2 per second) keeps COLMAP's matching cost near-linear in capture length, and checkpointed stages avoid reprocessing.
- Pipeline unit tests (pytest, with a synthetic apartment generator for offline tests) and web tests (A*, turn instructions, polygon maths) guard regressions.
- Measured figures for the incubation-centre scene (single 56.8 s portrait video, laptop RTX 4050 6 GB):
  - Keyframes: 293 extracted at 5.5 fps (1024 px); 65 registered by pycolmap SfM (largest of 7 sub-models, 6,660 points, 0.52 px mean reprojection error), SfM ~155 s on CPU.
  - Training: Brush 3DGS, 20,000 steps on the laptop GPU; 430,487 raw Gaussians.
  - Cleaning: 191,378 Gaussians kept after floater/transparency/bounds filtering; web bundle `scene.spz` 4.7 MB, navgrid 35.7 KB, packaging 9 s.
- Honest assessment: the low-texture interior and fast pans meant only 22% of frames registered, so the Round 1 scene covers part of the room and metric scale comes from the phone-height assumption (no ArUco markers in this capture). The revised SOP (slower pans, more overlap, printed markers) addresses this for the next capture.

7. Limitations and Future Work

Limitations: splats have no explicit surface mesh, so collision relies on the derived occupancy grid; mirrors, glass and blank walls degrade SfM and training; training needs a CUDA GPU (a Colab notebook is the fallback); multi-floor spaces and very large venues need more capture planning.

Future work: AR on-site navigation using the floor ArUco markers as anchors (already exported in `scene.json`) with live direction arrows through WebXR or the phone camera; semi-automatic POI suggestion from the splat; incremental re-capture of changed zones; multi-floor stitching.

8. Credits and Licences

Component	Use	Licence
3D Gaussian Splatting (Kerbl, Kopanas, Leimkuehler, Drettakis, SIGGRAPH 2023)	Core reconstruction method	Research paper (cited)
gsplat (nerfstudio-project)	3DGS training	Apache-2.0
Brush	Windows fallback trainer	Apache-2.0

COLMAP / pycolmap (incl. GLOMAP global mapper)	Structure from Motion	BSD
OpenCV (ArUco, image ops)	Marker detection, sharpness	Apache-2.0
NumPy, SciPy	Alignment, KD-tree, grid maths	BSD
ffmpeg	Frame extraction (external tool)	LGPL/GPL
PlayCanvas splat-transform	PLY to SPZ conversion	MIT
Spark.js	Splat rendering in Three.js	MIT
Three.js	3D rendering	MIT
React, Vite	UI framework, build tooling	MIT
Fuse.js	Fuzzy POI search	Apache-2.0
Polycam / Luma AI / Scaniverse	Optional alternative capture apps	Free-tier apps under their own terms

All other code in the repository is original work by the team.